



# Object Oriented Programming CSC2071

## Learning Session # 02

**Muhammad Shahid**

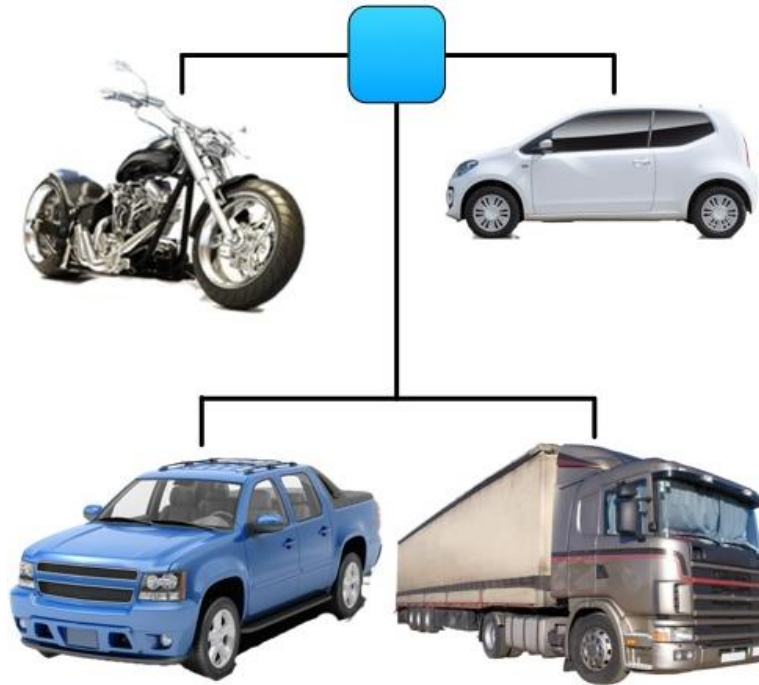
Department of Computer Science  
National Textile University

# Last Learning Session - Recap

- Language Basics
  - Console Applications and basic Inputs/Outputs
  - Operators (Relational & Conditional)
  - Ternary Operator [if, if-else, switch]
  - Iterations [while, do-while, for, foreach loops]
- Methods
  - Call By Values & Call By Reference
  - Method Overloading
- Arrays & Multidimensional Arrays
  - Arrays as Arguments & as Return values

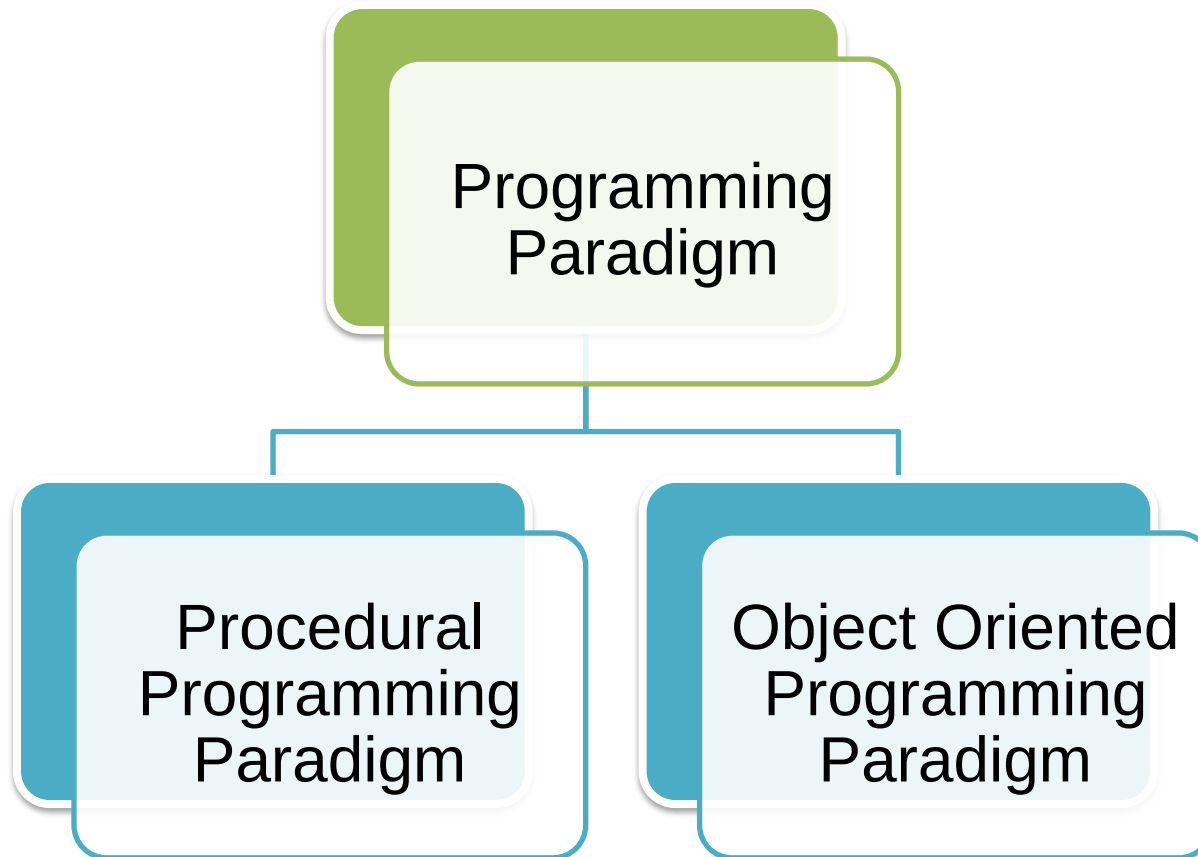
# Learning Session - Objectives

- Object-Orientation
  - What is Object-Orientation?
  - Procedural Programming Paradigm
  - Object Oriented Programming Paradigm
  - Benefits of Object-Orientation
- Object-Oriented Principles
  - Encapsulation
  - Abstraction
  - Inheritance
  - Polymorphism



# Object-Orientation

# Programming Paradigm



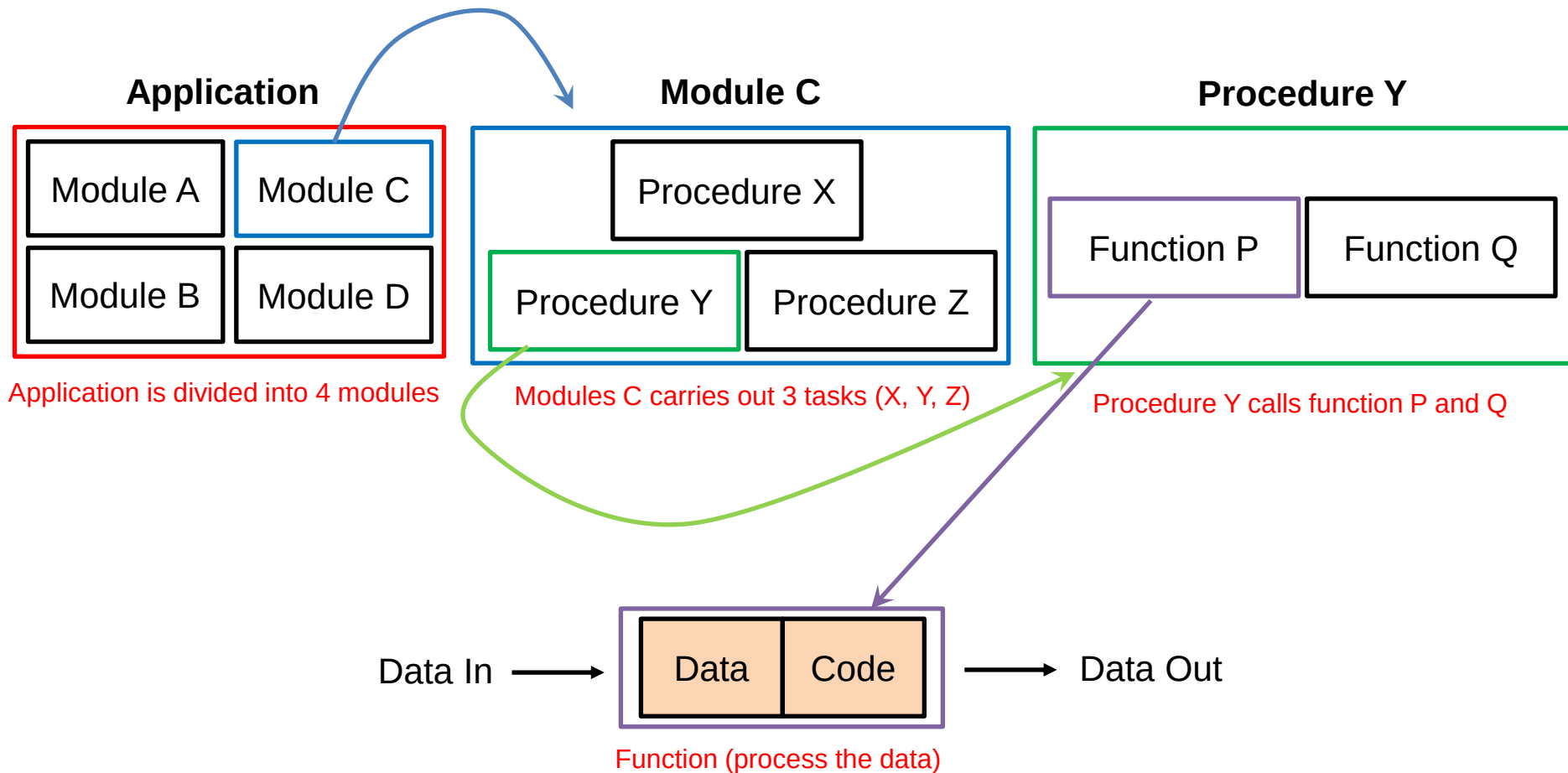
# Procedural Programming Paradigm

- In this paradigm (functional paradigm) the programming unit is a **function**
- Program is divided into **self-contained** and **maintainable** parts called '**Modules**'
- A module contains **procedures-self-contained** code that carries out a **single task**

# Procedural Programming Paradigm

- A **function** is a self-contained code returning value to the calling procedure
- All computations are done by applying functions
- Procedural paradigm separates **data** of program from **instructions** that manipulate the data

# Procedural Programming Paradigm





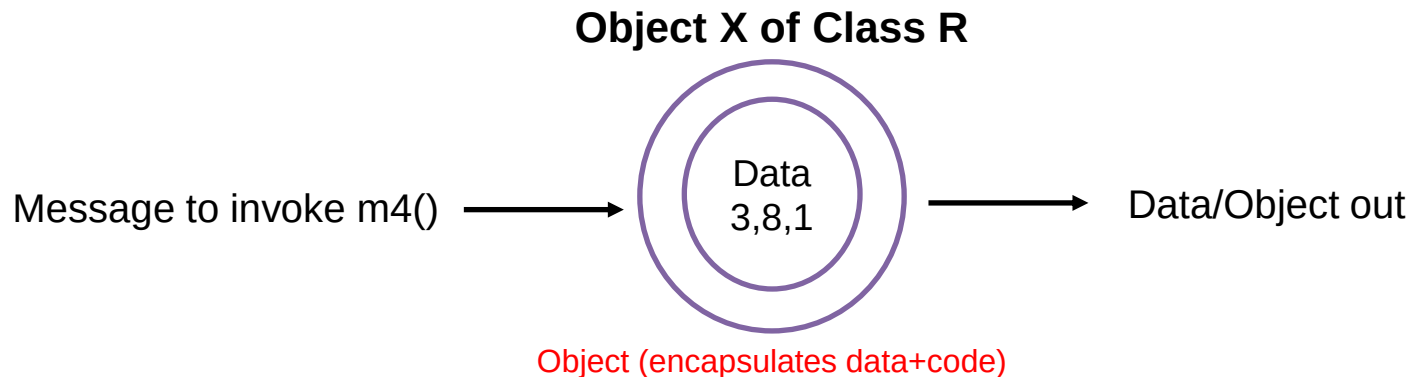
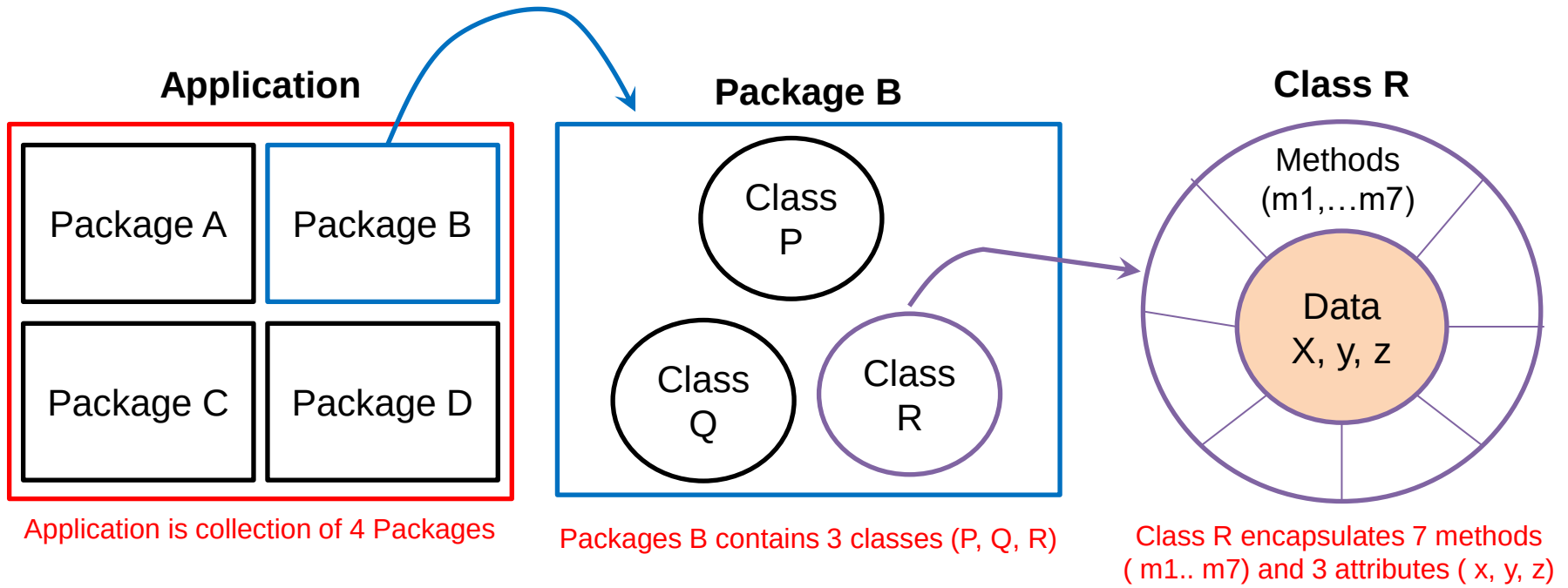
# OO Programming Paradigm

- OO Paradigm describes a system as it exists in **real life** based on **interactions** among **real objects**
- Application is modeled as **collection** of **related objects** that **interact** and do the **work** (task)
- The programming unit is a **class** representing **collection** of **similar real world objects**

# OO Programming Paradigm

- Programming starts with **abstraction** of useful **real world objects** and **classes**
- Application is **divided** into multiple **packages**
- OOP binds together **data** and the **instructions** that manipulate the data into a class

# OO Programming Paradigm



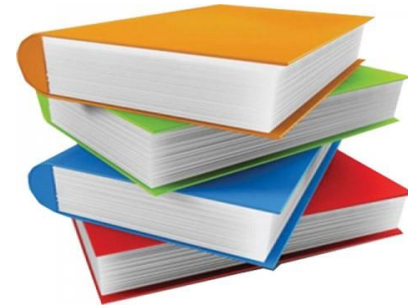
# What is Object-Orientation?

- A technique for system modeling
- OO model consists of several interacting objects

# What is a Model?

- A model is an abstraction of something
- Purpose is to understand the product before developing it

# Object-Oriented Model - Example



**Academic System**

# Object-Oriented Model - Example



## Highway System

# Object-Oriented Model - Example



## Computer System



# Benefits of Object-Orientation

- **Modularity**

- **Source code** for an object can be **maintained independently** in a class. Once created, an object can be easily passed around inside the system.

- **Code Re-use**

- If an **object already exists**, you can **use** that object in the **program**. This allows specialists to implement/test/debug complex, task-specific objects, which you can then trust to run in your own code.

# Benefits of Object-Orientation

- **Information-hiding**

- By interacting only with an object's methods, the details of its **internal implementation** remain **hidden** from the **outside world**

- **Plug-ability and debugging ease**

- If a particular object turns out to be **problematic**, you can **simply remove** it from the application and plug in a different object as its replacement. This is analogous to fixing mechanical problems in the real world.



# Object-Oriented Principles

# Object-Oriented Principles

**1. Encapsulation**

**2. Abstraction**

**3. Inheritance**

**4. Polymorphism**

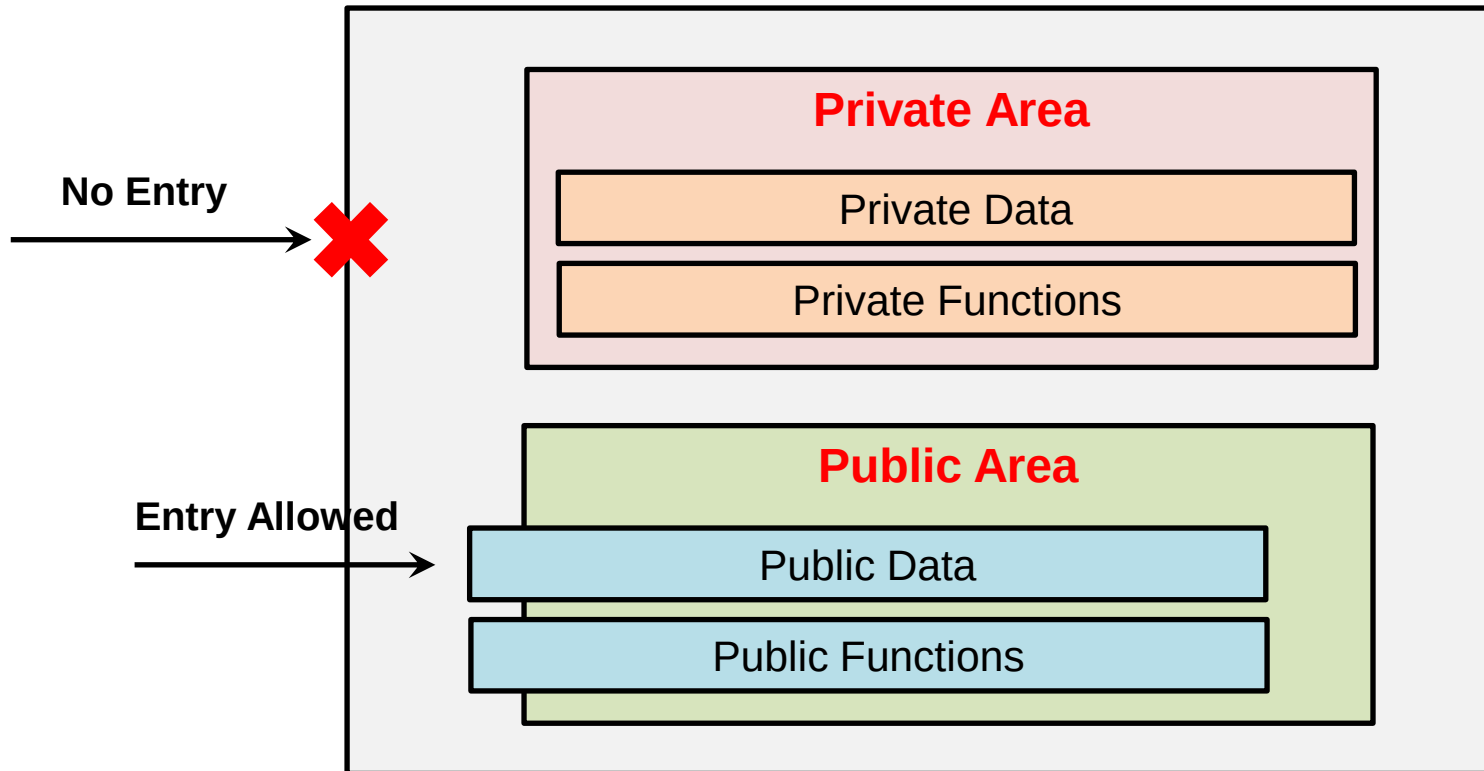
# OO Principles - Encapsulation

- Encapsulation is the mechanism that **binds together** the **code** and the **data** it manipulates, and **keeps both safe** from **outside interference** and **misuse**
- There are two important aspects of encapsulation:
  - 1) **Access restriction** - preventing one object from accessing another's internal state
  - 2) **Namespaces/scopes** - allowing the same name to have different meanings in different contexts

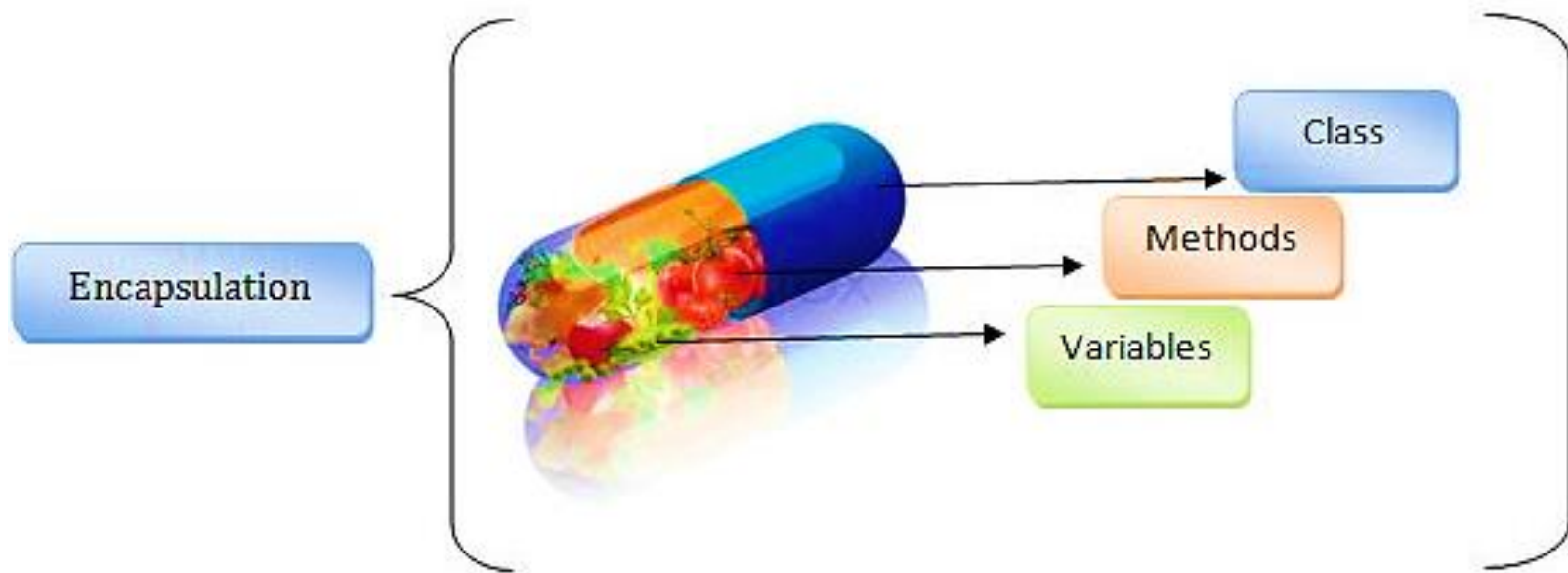
# OO Principles - Encapsulation

- Encapsulation mechanisms are **essential** for **reducing couplings** between software components
- Many encapsulation mechanisms originated with **non-object-oriented languages**
- Object-oriented languages **add additional encapsulation** mechanisms

# OO Principles - Encapsulation



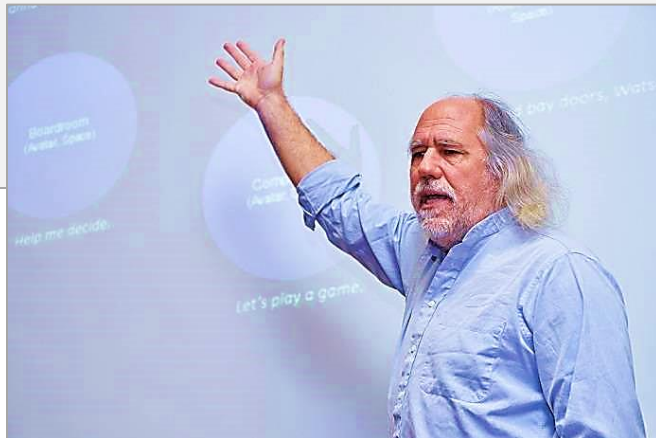
# OO Principles - Encapsulation





# OO Principles - Abstraction

- “An abstraction denotes the **essential characteristics** of an **object** that **distinguish** it from all **other** kinds of **object** and thus provide crisply defined **conceptual boundaries**, relative to the **perspective** of the **viewer**.”



[G. Booch]

# OO Principles - Abstraction

- Abstraction is the **development** of **classes**, **objects**, types in terms of their **interfaces** and **functionality**, **instead** of their **implementation details**
- Abstraction denotes a **model**, a **view**, or some other focused **representation** for an **real world object**

# OO Principles - Abstraction



# Diff. b/w Encapsulation & Abstraction

- **Abstraction** means to hide the unnecessary data from the user. The user only needs the required functionality or the output according to his requirements.
- **Encapsulation** is simply combining the data members and functions into a single entity called an object.

# Diff. b/w Encapsulation & Abstraction

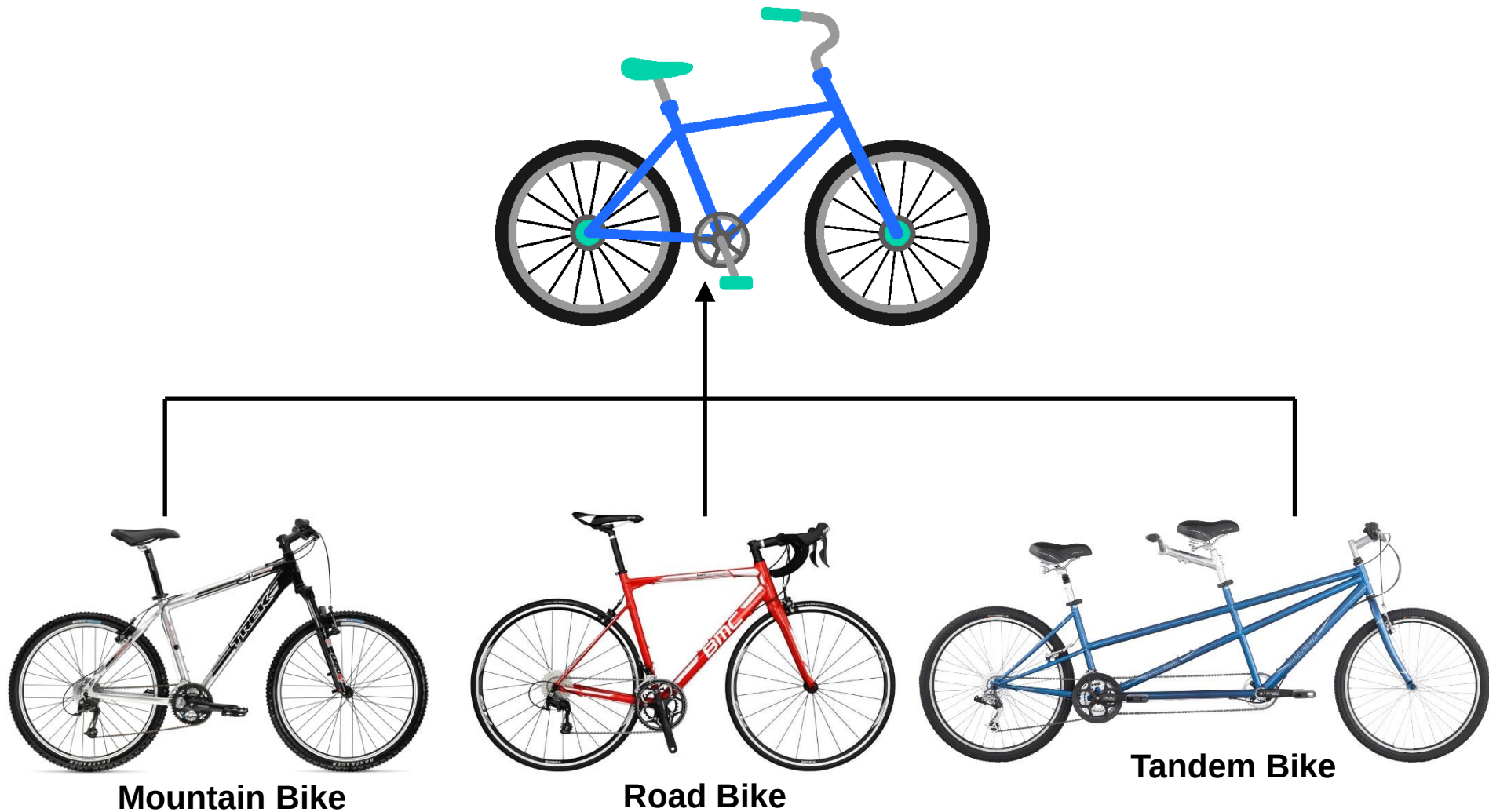


Abstraction is achieved through encapsulation

# OO Principles - Inheritance

- Inheritance is the process by which one **object acquires** the **properties** of **another object**
- By use of inheritance, an object need only define all of its characteristics that make it unique within its class, it can **inherit** its **general attributes** from its **parent**
- Object-oriented programming allows classes to **inherit commonly used state** and **behavior** from other classes

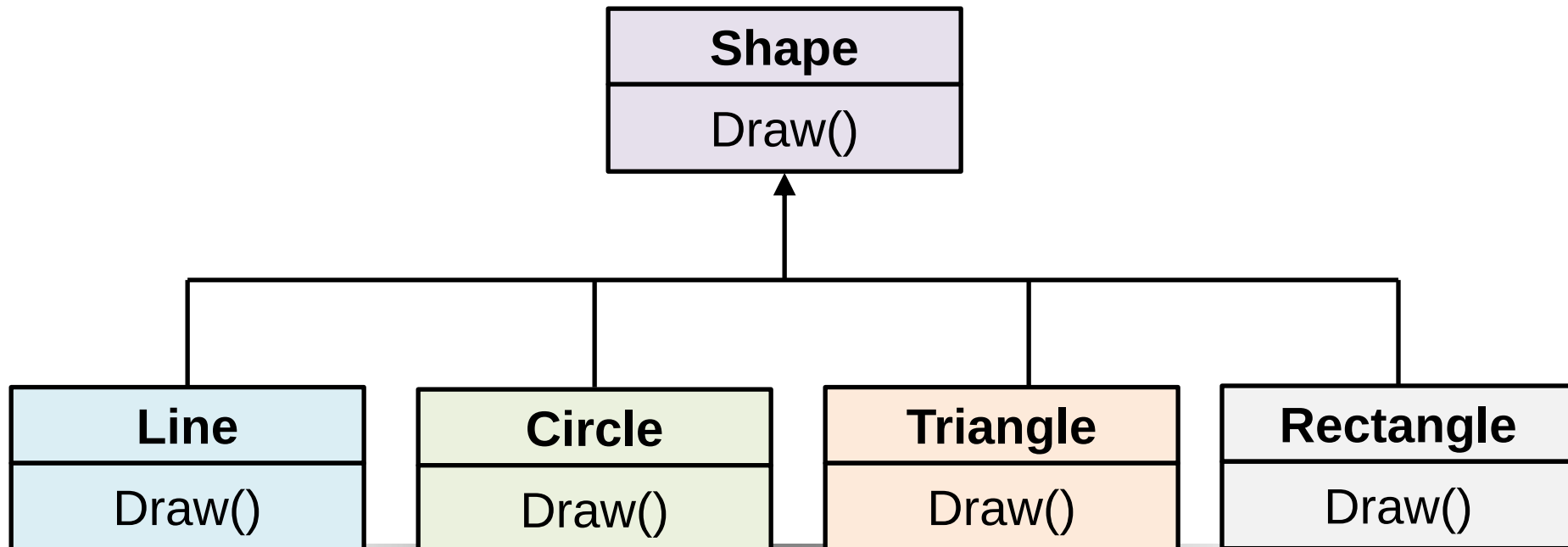
# OO Principles - Inheritance





# OO Principles - Polymorphism

- Polymorphism (from **Greek**, meaning '**many forms**') is a feature that allows **same interface** to be used for a **multiple class** of actions depending on context





# Learning Session - Recap

- Object-Orientation
  - What is Object-Orientation?
  - Procedural Programming Paradigm
  - Object Oriented Programming Paradigm
  - Benefits of Object-Orientation
- Object-Oriented Principles
  - Encapsulation
  - Abstraction
  - Inheritance
  - Polymorphism

# Questions

